AI 计算系统
实验操作手册

实验序号：四

实验名称：基于 MindSpore 框架
实现 VGG17 训练与验证

教　　师：朱光辉

学　　校：南京大学

时　　间：2022 年 8 月 23 日

四 基于MindSpore框架实现VGG17训练与验证

4.1 实验目标

- 熟悉和使用MindSpore框架和ModelArts，熟悉MindSpore常见API的使用方法，熟悉ModelArts一站式模型训练和部署平台。
- 基于MindSpore框架构建VGG17网络。利用花卉数据集上完成模型训练（训练平台：ModelArts，可采用昇腾910芯片进行训练）。模型训练完成后，对模型进行保存。
- 基于昇腾310推理芯片作为计算平台，利用MindSpore框架导入训练好的模型，并在花卉测试数据集对构建的模型进行推理验证，输出推理性能以及测试集正确率。
- 本实验希望借助MindSpore帮助学生熟悉使用深度学习框架，感受框架封装基本操作的便捷。

4.2 背景介绍

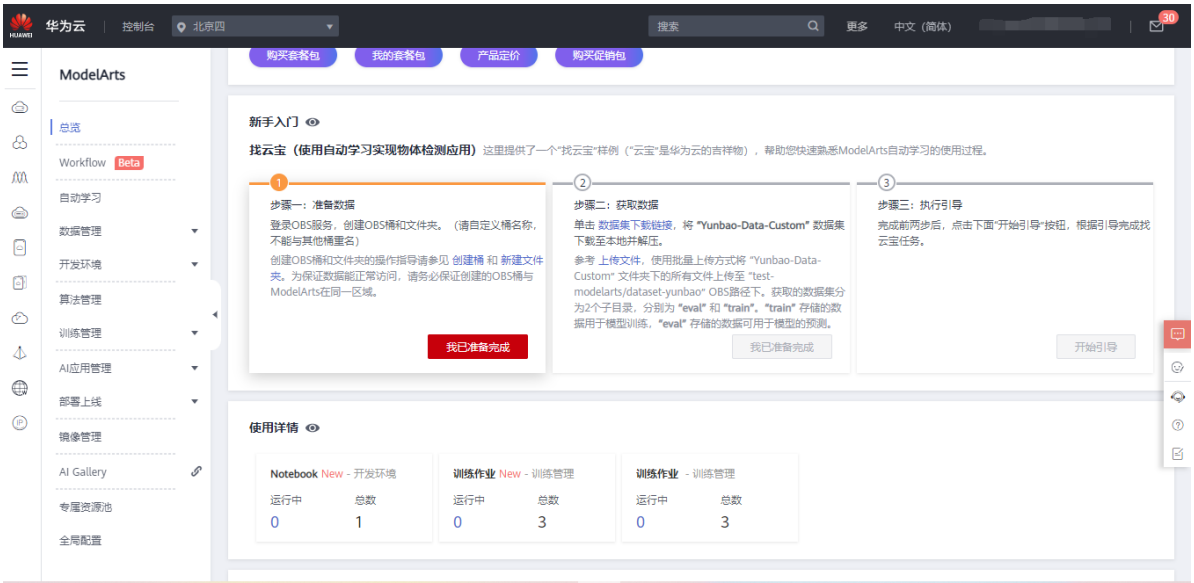
4.2.1 MindSpore

MindSpore是华为推出的深度学习框架，拥有自动微分、并行加持，一次训练，可多场景部署的特性。为全场景AI的模型开发、模型运行、模型部署提供端到端能力。 MindSpore支持深度学习算法在Ascend、GPU、CPU等硬件平台上开发和部署。

- 如需查看详情，请参见如下资源：
  - MindSpore教程
  - MindSpore Python API

4.2.2 ModelArts

ModelArts是面向AI开发者的一站式开发平台，包含AI开发的各个环节，包括数据处理、算法开发、模型训练、模型部署都可以在ModelArts上完成。从技术上看，ModelArts底层支持各种异构计算资源，开发者可以根据需要灵活选择使用，而不需要关心底层的技术。同时，ModelArts支持Tensorflow、PyTorch、MindSpore等主流开源的AI开发框架。



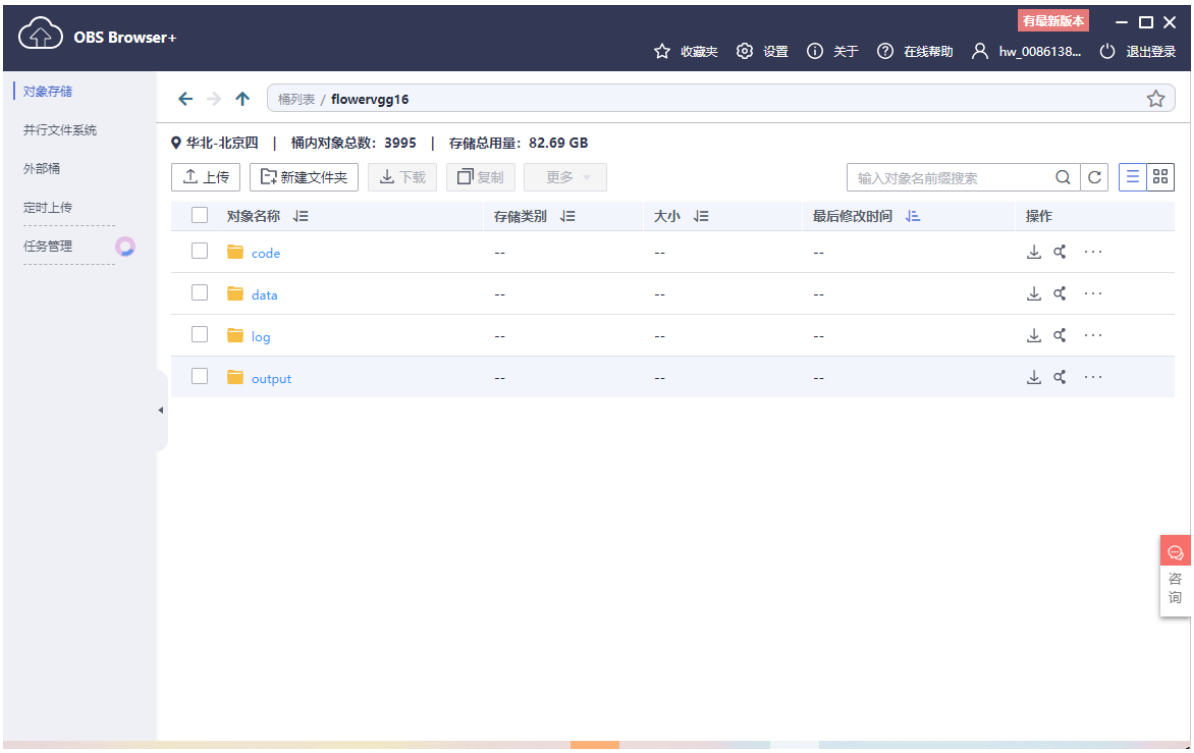
4.2.3.1 OBS桶

ModelArts 使用对象存储服务（Object Storage Service，简称OBS）进行数据存储及模型的备份。OBS的基本组成是桶和对象。桶是OBS中存储对象的容器，对象是代码和数据集文件，每个桶都有自己的存储类别、访问权限等属性。ModelArts通过访问OBS桶中对应的文件来访问数据。因此，在开始训练任务之前，需要将数据和训练脚本上传至OBS。这里使用OBS Brwoser+工具管理OBS。

上传步骤：

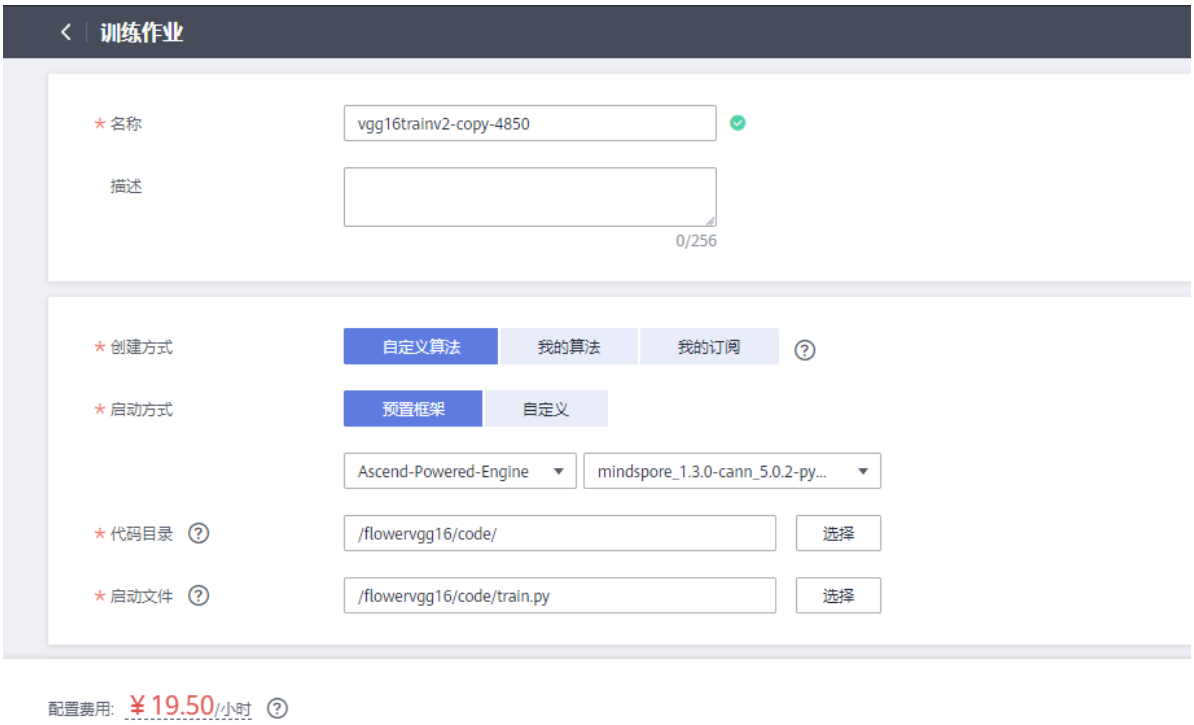
- 打开OBS Browser+
- 上传代码文件：点击创建的OBS桶 —> 点击对象 —> 创建code文件夹 —> 上传文件 —>完成代码文件上传
- 上传数据集文件：点击创建的OBS桶 —> 创建data文件夹 —> 上传数据集文件 —> 完成数据集上传
- 创建log文件夹 —>存放打印日志
- 创建output文件夹 —> 存放输出的文件

  例如这里桶的名称为flowervgg16，分别有4个文件夹。



4.2.3.1 ModelArts的训练和评估页面

在ModelArts界面中选择训练管理 —> 训练作业 —> 创建作业



4.2.3 VGG17网络

本次实验延续之前实验中使用的VGG16网络并进行一些变换提出VGG17网络，网络结构如下。VGG17网络的特点是：

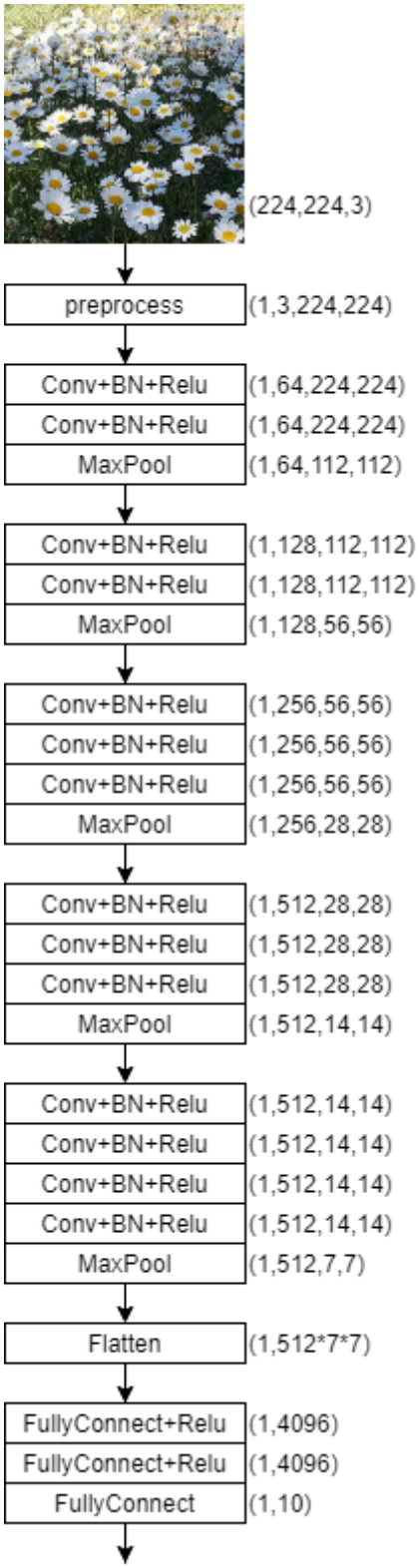
- 在VGG中，使用了3个3x3卷积核来代替7x7卷积核，使用了2个3x3卷积核来代替5*5卷积核，相比AlexNet中的较大卷积核（11x11，7x7，5x5），VGG网络层数更深，提升了网络性能。
- 池化层均采用相同的池化核参数，stride=2，
- 模型由若干卷积层和池化层堆叠的方式构成

注：在构造网络时，还需要考虑BN(Batch Normalization)层和Relu层（BN层可以提高网络训练稳定性，Relu层是非线性激活层）。此外为了提高网络鲁棒性，加入了dropout层。

算子	类型	输入通道数	输出通道数	窗口大小	边界扩充大小	步长	输出张量的高度和宽度
layer1_conv1	卷积	3	64	3	1	1	224x224
layer1_conv2	卷积	64	64	3	1	1	224x224
layer1_maxpool	最大池化	64	64	2	-	2	112x112
layer2_conv1	卷积	64	128	3	1	1	112x112
layer2_conv2	卷积	128	128	3	1	1	112x112

算子	类型	输入通道数	输出通道数	窗口大小	边界扩充大小	步长	输出张量的高度和宽度
layer2_maxpool	最大池化	128	128	2	-	2	56x56
layer3_conv1	卷积	128	256	3	1	1	56x56
layer3_conv2	卷积	256	256	3	1	1	56x56
layer3_conv3	卷积	256	256	3	1	1	56x56
layer3_maxpool	最大池化	256	256	2	-	2	28x28
layer4_conv1	卷积	256	512	3	1	1	28x28
layer4_conv2	卷积	512	512	3	1	1	28x28
layer4_conv3	卷积	512	512	3	1	1	28x28
layer4_maxpool	最大池化	512	512	2	-	2	14x14
layer5_conv1	卷积	512	512	3	1	1	14x14
layer5_conv2	卷积	512	512	3	1	1	14x14
layer5_conv3	卷积	512	512	3	1	1	14x14
layer5_conv4	卷积	512	512	3	1	1	14x14
layer5_maxpool	最大池化	512	512	2	-	2	7x7
flatten	扁平化	-	-	-	-	-	-
fullyconnect1	全连接	25088	4096	-	-	-	-
fullyconnect2	全连接	4096	4096	-	-	-	-
fullyconnect3	全连接	4096	4	-	-	-	-

VGG17的结构图如下：



4.3 实验环境

- 平台
  - ModelArts AI开发平台（Ascend 910训练+Ascend 310推理）

    [ModelArts平台的使用文档]
- 依赖
  - [MindSpore]
  - numpy

4.4 实验步骤

本实验的代码目录介绍：

```
├── vgg17
    ├── data                                    // 数据集文件夹
    ├── code                                    // 代码文件夹
        ├── ascend310_infer                     // Ascend310 推理目录
        ├── infer                               // MindX 推理目录
        ├── model_utils
        |   ├── __init__.py                 // 初始化文件
        |   ├── config.py                   // 参数配置
        |   ├── device_adapter.py           // ModelArts的设备适配器
        |   ├── local_adapter.py            // 本地适配器
        |   └── moxing_adapter.py           // ModelArts的模型适配器
        ├── scripts
        |   ├── run_distribute_train.sh           // Ascend 分布式训练shell脚本
        |   ├── run_distribute_train_gpu.sh       // GPU 分布式训练shell脚本
        |   ├── run_eval.sh                       // Ascend 验证shell脚本
        |   ├── run_infer_310.sh                  // Ascend310 推理shell脚本
        ├── src
        |   ├── utils
        |   |   ├── logging.py                  // 日志格式设置
        |   |   ├── sampler.py                  // 为数据集创建采样器
        |   |   ├── util.py                     // 工具函数
        |   |   ├── var_init.py                 // 网络参数init方法
        |   ├── crossentropy.py                 // 损失计算
        |   ├── dataset.py                      // 创建数据集
        |   ├── linear_warmup.py                // 线性学习率
        |   ├── warmup_cosine_annealing_lr.py   // 余弦退火学习率
        |   ├── warmup_step_lr.py               // 单次或多次迭代学习率
        |   ├──vgg.py                           // VGG架构
        ├── train.py                            // 训练脚本
        ├── eval.py                             // 评估脚本
        ├── postprocess.py                      // 后处理脚本
        ├── preprocess.py                       // 预处理脚本
        ├── mindspore_hub_conf.py               // mindspore hub 脚本
        ├── flowerphotos_config.yaml            // flowerphotos 配置文件
        ├── export.py                           // 模型格式转换脚本
```

4.4.1 MindSpore常用函数介绍

mindspore.nn模块中也提供了一系列的算子接口，主要是用于构建神经网络的预定义构建块或计算单元。在构建VGG网络模型时，我们主要用到了以下API，如下表所示。

MindSpore中的常用算子接口

算子接口名称	功能描述	参数介绍

算子接口名称	功能描述	参数介绍
mindspore.nn. Conv2d	对输入Tensor计算二维卷积	**in_channels** (int) : Conv2d层输入Tensor的空间维度 **out_channels** (dict) : Conv2d层输出Tensor的空间维度 **kernel_size** (Union[int, tuple[int]])： 卷积核的高和宽 **padding** (Union[int, tuple[int]])： 边界扩充。若padding 类型为int，则四个方向上的填充都等于 padding 。若 padding 为包含4个int的tuple，则上、下、左、右的填充分别等于 padding[0] 、 padding[1] 、 padding[2] 和 padding[3] 。值应该要大于等于0，默认值：0 **stride** (Union[int, tuple[int]])：卷积步长，默认值：1 **pad_mode** (str)：填充模式，可选“same”，“valid”，“pad”，默认值：“same”。“same”表示保留边界处的卷积结果，“valid”表示只进行有效卷积，对边界不处理 **dilation** (Union[int, tuple[int]])：指二维卷积扩张的尺寸 **has_bias** (bool)： Conv2d层是否添加偏置参数，若conv后接batchnorm层不添加偏置，反之添加，默认值：False **weight_init** (Union[Tensor, str, Initializer, numbers.Number])： 卷积核参数初始化的方法，可以是张量，字符串，初始化器或数字。当使用str时，可选"TruncatedNormal"，"Uniform"，“Zero”等分布。默认值：“normal”，即用正态分布的随机数初始化卷积层的权重参数 **bias_init** (Union[Tensor, str, Initializer, numbers.Number])： 偏置参数初始化的方法，默认值："zeros" **data_format** (str)： 数据格式的可选值有”NHWC”，”NCHW”。默认值：”NCHW”
mindspore.nn.ReLU	对输入Tensor做ReLU，返回的数据类型同输入Tensor	**x** (Tensor)： 用于计算ReLU的任意维度的Tensor。数据类型为number类型，包括float32,float64,int32,uint8,int16,int64,bfloat16,uint16,half,uint32,uint64和qint8
mindspore.nn.BatchNorm2d	对输入Tensor进行批归一化	**num_features** (int)： 指定输入Tensor的通道数量，输入Tensor的size为 (*N*,*C*,*H*,*W*)
mindspore.nn.Dropout	Dropout是一种正则化手段，该算子根据丢弃概率 1−*keep_prob1*，在训练过程中随机将一些神经元输出设置为0，通过阻止神经元节点间的相关性来减少过拟合	**keep_prob** (float) : 输入神经元保留率，数值范围在0到1之间。例如，rate=0.9，删除10%的神经元。默认值：0.5。 **dtype** (mindspore.dtype) : x 的数据类型。默认值：float32。
mindspore.nn.SoftmaxCrossEntropyWithLogits	计算预测值和真实值的交叉熵	**logits** (Tensor)： 真实值Tensor，shape (N, C),数据类型为float16或float32 **labels** (Tensor)： 预测值Tensor，shape (N,)。如果 sparse 为True，则 labels 的类型为int32或int64。否则，labels 的类型与 logits 的类型相同。
mindspore.nn. MaxPool2d	对输入的多维数据进行二维的最大池化运算	**kernel_size** (Union[int, tuple[int]])： 池化核的高和宽。默认值：1 **stride** (Union[int, tuple[int]])： 池化操作的移动步长。默认值：1 **pad_mode** (str)： 指定池化填充模式，取值为”same”或”valid”。默认值：”valid” **data_format** (str)： 输入数据格式可为 ”NHWC“ 或 ”NCHW“。默认值：’NCHW’

关于更多API请参考MindSpore的API文档 https://www.mindspore.cn/docs/zh-CN/r1.8/index.html

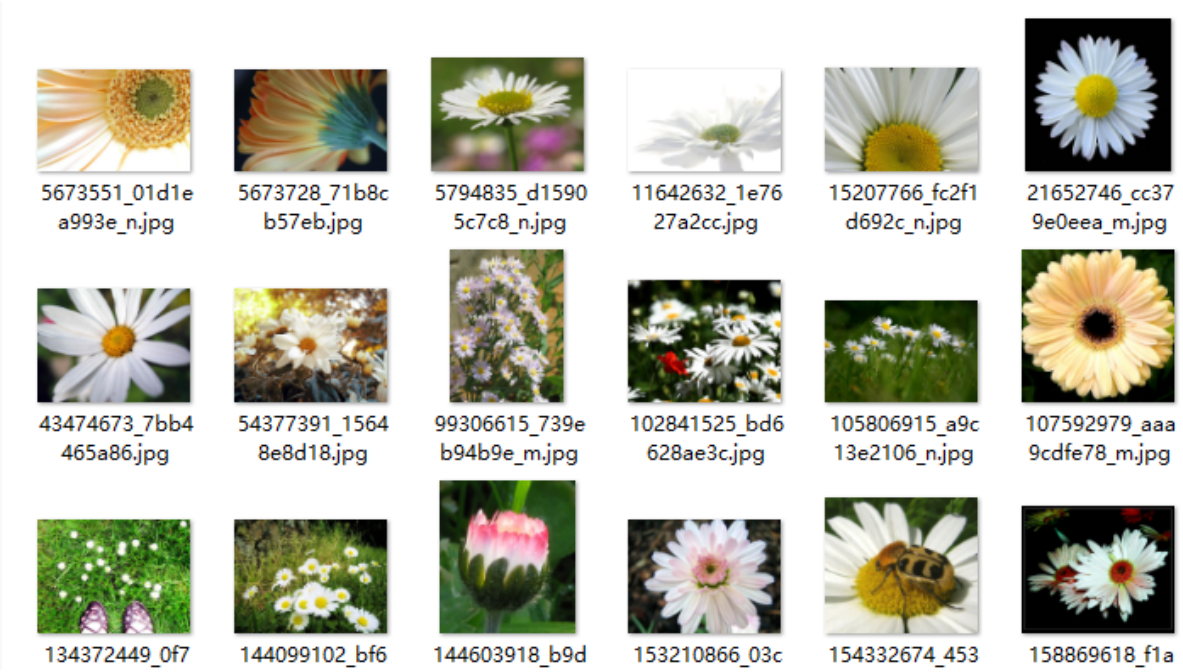
4.4.2 数据加载模块

4.4.2.1 数据集加载

本实验顺承上次实验，采用花卉数据集，利用mindspore.dataset类下的`ImageFolderDataset`加载图片数据，同一个文件夹中的所有图片将被分配相同的label。对应的数据集目录为：

```
└─data
    ├─test
    |  ├─daisy
    |  ├─dandelion
    |  ├─roses
    |  ├─sunflowers
    |  └─tulips
    └─train
       ├─daisy
       ├─dandelion
       ├─roses
       ├─sunflowers
       └─tulips
```

例如，（./data/train/daisy）目录下的图片如下所示



想了解更多的数据加载方式，请参考 MindSpore深度学习框架教程 和 MindSpore数据处理教程 。

4.4.2.2 数据增强

在图片训练中，尤其在数据集较小的情况下，用户可以通过一系列的数据增强操作对图片进行预处理，从而丰富了数据集。MindSpore的`mindspore.dataset.vision`模块提供一系列图像数据增强的操作。以下介绍几种常见的数据增强方法：

- **RandomCrop**

  对输入图像进行在随机位置的裁剪。

  参数说明：

  - `size`：裁剪图像的尺寸。
  - `padding`：填充的像素数量。
  - `pad_if_needed`：原图小于裁剪尺寸时，是否需要填充。
  - `fill_value`：在常量填充模式时使用的填充值。
  - `padding_mode`：填充模式。
- **RandomHorizontalFlip**

对输入图像进行随机水平翻转。

  - `prob`：单张图片发生翻转的概率。

- **HWC2CHW**

  将输入图像的shape从 <H, W, C> 转换为 <C, H, W>。 如果输入图像的shape为 <H, W> ， 图像将保持不变。

- **Resize**

  对输入图像进行缩放

  - `size`：缩放的目标大小。
  - `interpolation`：缩放时采用的插值方式。

数据加载模块的对应文件为（./code/src/dataset.py），代码如下：

```
def vgg_create_dataset(data_home, image_size, batch_size, rank_id=0,
rank_size=1, training=True):
    """Data operations."""
    data_dir = os.path.join(data_home, "train")
    if not training:
        data_dir = os.path.join(data_home, "test")

    data_set = de.ImageFolderDataset(data_dir,
                                     class_indexing=
{'daisy':0,'dandelion':1,'roses':2,'sunflowers':3,'tulips':4},
                                     shuffle=False, num_shards=rank_size,
shard_id=rank_id)

    #数据增强的方法
    transform_img = vision.RandomCropDecodeResize([224,224], scale=(0.08, 1.0),
                                                  ratio=(0.75, 1.333))  # 改变尺寸
    changeswap_op = vision.HWC2CHW()
    type_cast_op = C.TypeCast(mstype.float32)
    random_horizontal_op = vision.RandomHorizontalFlip()

    # map操作将指定函数操作于数据集的指定列数据
    data_set = data_set.map(input_columns="image", operations=transform_img)
    data_set = data_set.map(input_columns="image", operations=type_cast_op)
    data_set = data_set.map(input_columns="image",
operations=random_horizontal_op)
    data_set = data_set.map(input_columns="image", operations=changeswap_op)

    # shuffle来进行数据集的混洗
    data_set = data_set.shuffle(buffer_size=data_set.get_dataset_size())

    # 连续 batch_size 条数据合并为一个批处理数据
    data_set = data_set.batch(batch_size=batch_size, drop_remainder=True)

    return data_set
```

4.4.3 网络结构模块（需要自实现）

本模块需要利用MindSpore.nn相关API完整搭建VGG17网络结构（网络结构对应文件相对路径为：(./code/src/vgg.py）。可参考git仓源码。需熟悉使用MindSpore构建网络的相关API，可参考4.4节API接口介绍。

在利用MindSpore构建网络时，需要继承`mindspore.nn.Cell`类，并重写`__init__`方法和`construct`方法。

- 在`__init__`函数中声明网络各个层和算子的定义。
- 在`construct`函数中完成网络的构造。

网络结构模块对应文件为（./code/src/vgg.py）。请同学按照VGG17的网络结构（如4.4.2小结所示的VGG17网络），以层级顺序搭建的方式搭建VGG17网络（如下图的参考示例中的example）。

```
# Copyright 2020 Huawei Technologies Co., Ltd
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
# ============================================================================
"""
Image classifiation.
"""
import mindspore.nn as nn


class Vgg(nn.Cell):
    """
    TO-DO.

    参数:
        num_classes (int): Class numbers. Default: 5.
        phase (int): 指定是训练/评估阶段

    返回值:
        Tensor, infer output tensor.

    example:
        self.layer1_conv1 = nn.Conv2d(in_channels=3, out_channels=64,
kernel_size=3,weight_init='XavierUniform')
        self.layer1_bn1 = nn.BatchNorm2d(num_features=64)
        self.layer1_relu1 = nn.LeakyReLU()

    """
    def __init__(self, num_classes=5, args=None, phase="train"):
        super(Vgg, self).__init__()
```

```
        dropout_ratio = 0.5
        if not args.has_dropout or phase == "test":
            dropout_ratio = 1.0

        self.layer1_conv1 = nn.Conv2d(in_channels=3, out_channels=64,
kernel_size=3,weight_init='XavierUniform')
        self.layer1_bn1 = nn.BatchNorm2d(num_features=64)
        self.layer1_relu1 = nn.ReLU()
        self.layer1_conv2 = nn.Conv2d(in_channels=64, out_channels=64,
kernel_size=3,weight_init='XavierUniform')
        self.layer1_bn2 = nn.BatchNorm2d(num_features=64)
        self.layer1_relu2 = nn.ReLU()
        self.layer1_maxpool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.layer2_conv1 = nn.Conv2d(in_channels=64, out_channels=128,
kernel_size=3,weight_init='XavierUniform')
        self.layer2_bn1 = nn.BatchNorm2d(num_features=128)
        self.layer2_relu1 = nn.ReLU()
        self.layer2_conv2 = nn.Conv2d(in_channels=128, out_channels=128,
kernel_size=3,weight_init='XavierUniform')
        self.layer2_bn2 = nn.BatchNorm2d(num_features=128)
        self.layer2_relu2 = nn.ReLU()
        self.layer2_maxpool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.layer3_conv1 = nn.Conv2d(in_channels=128, out_channels=256,
kernel_size=3,weight_init='XavierUniform')
        self.layer3_bn1 = nn.BatchNorm2d(num_features=256)
        self.layer3_relu1 = nn.ReLU()
        self.layer3_conv2 = nn.Conv2d(in_channels=256, out_channels=256,
kernel_size=3,weight_init='XavierUniform')
        self.layer3_bn2 = nn.BatchNorm2d(num_features=256)
        self.layer3_relu2 = nn.ReLU()
        self.layer3_conv3 = nn.Conv2d(in_channels=256, out_channels=256,
kernel_size=3,weight_init='XavierUniform')
        self.layer3_bn3 = nn.BatchNorm2d(num_features=256)
        self.layer3_relu3 = nn.ReLU()
        self.layer3_maxpool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.layer4_conv1 = nn.Conv2d(in_channels=256, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer4_bn1 = nn.BatchNorm2d(num_features=512)
        self.layer4_relu1 = nn.ReLU()
        self.layer4_conv2 = nn.Conv2d(in_channels=512, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer4_bn2 = nn.BatchNorm2d(num_features=512)
        self.layer4_relu2 = nn.ReLU()
        self.layer4_conv3 = nn.Conv2d(in_channels=512, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer4_bn3 = nn.BatchNorm2d(num_features=512)
        self.layer4_relu3 = nn.ReLU()
        self.layer4_maxpool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.layer5_conv1 = nn.Conv2d(in_channels=512, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer5_bn1 = nn.BatchNorm2d(num_features=512)
```

```
        self.layer5_relu1 = nn.ReLU()
        self.layer5_conv2 = nn.Conv2d(in_channels=512, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer5_bn2 = nn.BatchNorm2d(num_features=512)
        self.layer5_relu2 = nn.ReLU()
        self.layer5_conv3 = nn.Conv2d(in_channels=512, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer5_bn3 = nn.BatchNorm2d(num_features=512)
        self.layer5_relu3 = nn.ReLU()
        self.layer5_conv4 = nn.Conv2d(in_channels=512, out_channels=512,
kernel_size=3,weight_init='XavierUniform')
        self.layer5_bn4 = nn.BatchNorm2d(num_features=512)
        self.layer5_relu4 = nn.ReLU()
        self.layer5_maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()

        self.fullyconnect1 = nn.Dense(512 * 7 * 7, 4096)
        self.relu_1 = nn.ReLU()
        self.dropout_1 = nn.Dropout(dropout_ratio)

        self.fullyconnect2 = nn.Dense(4096, 4096)
        self.relu_2 = nn.ReLU()
        self.dropout_1 = nn.Dropout(dropout_ratio)

        self.fullyconnect3 = nn.Dense(4096, num_classes)


    def construct(self, x):
        x  =  self.layer1_conv1(x)
        x  =  self.layer1_bn1(x)
        x  =  self.layer1_relu1(x)
        x  =  self.layer1_conv2(x)
        x  =  self.layer1_bn2(x)
        x  =  self.layer1_relu2(x)
        x  =  self.layer1_maxpool(x)

        x  =  self.layer2_conv1(x)
        x  =  self.layer2_bn1(x)
        x  =  self.layer2_relu1(x)
        x  =  self.layer2_conv2(x)
        x  =  self.layer2_bn2(x)
        x  =  self.layer2_relu2(x)
        x  =  self.layer2_maxpool(x)

        x  =  self.layer3_conv1(x)
        x  =  self.layer3_bn1(x)
        x  =  self.layer3_relu1(x)
        x  =  self.layer3_conv2(x)
        x  =  self.layer3_bn2(x)
        x  =  self.layer3_relu2(x)
        x  =  self.layer3_conv3(x)
        x  =  self.layer3_bn3(x)
        x  =  self.layer3_relu3(x)
        x  =  self.layer3_maxpool(x)
```

```
        x  =  self.layer4_conv1(x)
        x  =  self.layer4_bn1(x)
        x  =  self.layer4_relu1(x)
        x  =  self.layer4_conv2(x)
        x  =  self.layer4_bn2(x)
        x  =  self.layer4_relu2(x)
        x  =  self.layer4_conv3(x)
        x  =  self.layer4_bn3(x)
        x  =  self.layer4_relu3(x)
        x  =  self.layer4_maxpool(x)

        x  =  self.layer5_conv1(x)
        x  =  self.layer5_bn1(x)
        x  =  self.layer5_relu1(x)
        x  =  self.layer5_conv2(x)
        x  =  self.layer5_bn2(x)
        x  =  self.layer5_relu2(x)
        x  =  self.layer5_conv3(x)
        x  =  self.layer5_bn3(x)
        x  =  self.layer5_relu3(x)
        x  =  self.layer5_conv4(x)
        x  =  self.layer5_bn4(x)
        x  =  self.layer5_relu4(x)
        x  =  self.layer5_maxpool(x)

        x = self.flatten(x)
        x = self.fullyconnect1(x)
        x = self.relu_1(x)
        x = self.dropout_1(x)
        x = self.fullyconnect2(x)
        x = self.relu_2(x)
        x = self.dropout_1(x)
        x = self.fullyconnect3(x)

        return x



def vgg17(num_classes=1000, args=None, phase="train", **kwargs):
    """
    生成VGG17网络实例
    参数:
        num_classes (int): 分类数
        args (namespace): 参数
        phase (str): 指定是训练/评估阶段
    返回:
        Cell, cell instance of Vgg17 neural network with Batch Normalization.

    参考如下:
        >>> vgg17(num_classes=5, args=args, **kwargs)
    """
    net = Vgg(num_classes=num_classes, args=args, phase=phase, **kwargs)
    return net
```

4.4.4 参数配置

在 (./code/flowerphotos_config.yaml) 中配置了VGG16的训练参数和评估参数。可自行尝试调整lr，batch_size，max_epoch等参数，提高模型精度。

```yaml
num_classes: 5                    # 数据集类数
lr: 0.01                          # 学习率
lr_init: 0.01                     # 初始学习率
lr_max: 0.1                       # 最大学习率
lr_epochs: '30,60,90,120'         # 基于变化lr的轮次
lr_scheduler: "step"              # 学习率模式
warmup_epochs: 5                  # 热身轮次数
batch_size: 64                    # 输入张量批次大小
max_epoch: 100                    # 训练轮次，只对训练有效
momentum: 0.9                     # 动量
weight_decay: 0.0005              # 权重衰减
loss_scale: 1.0                   # 损失放大
label_smooth: 0                   # 标签平滑
label_smooth_factor: 0            # 标签平滑因子
buffer_size: 10                   # 混洗缓冲区大小
image_size: '224,224'             # 图像大小
pad_mode: 'same'                  # conv2d的填充方式
padding: 0                        # conv2d的填充值
has_bias: False                   # conv2d是否有偏差
batch_norm: True                  # 在conv2d中是否有batch_norm
keep_checkpoint_max: 10           # 只保留最后一个keep_checkpoint_max检查点
initialize_mode: "XavierUniform"  # conv2d init模式
has_dropout: True                 # 是否使用Dropout层
```

4.4.5 训练与评估

MindSpore提供`mindspore.Model`用于模型训练和推理的API，Model会根据用户传入的参数封装可训练或推理的实例。

4.4.5.1 模型训练

使用`mindspore.Model.train`接口执行模型训练，`train`接口的常用参数如下：

- `epoch`：训练执行轮次，通常每个epoch都会使用全量数据集进行训练。
- `train_dataset`：一个训练数据集迭代器。
- `callbacks`：训练过程中需要执行的回调对象或者回调对象列表。

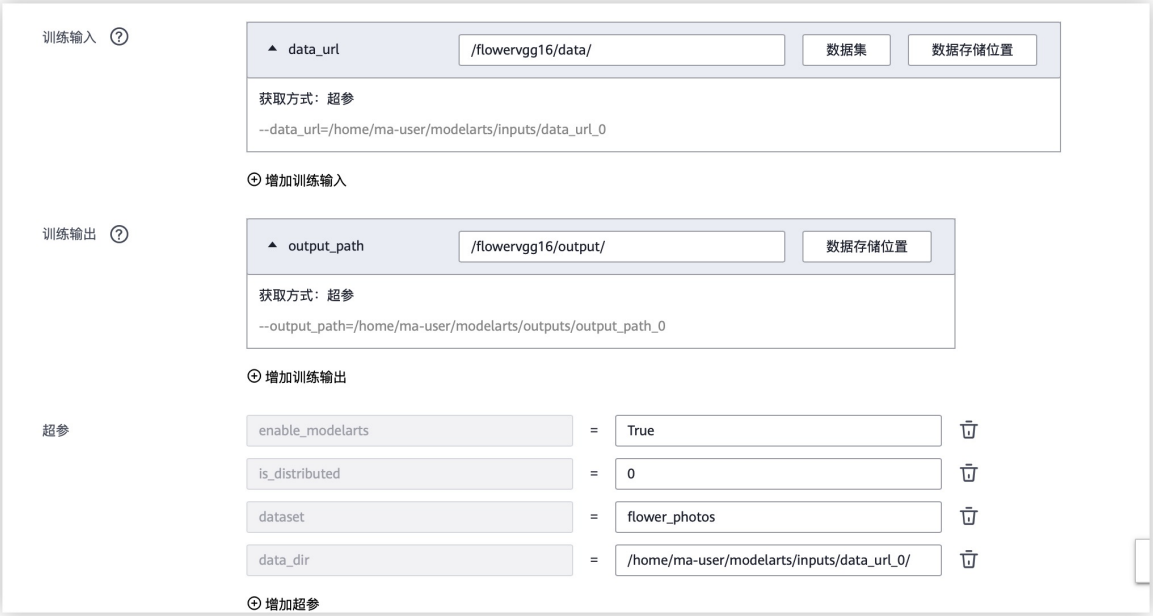
训练文件对应为（./code/train.py），相关代码如下：

```python
    # 定义神经网络
    network = vgg17(config.num_classes, config)

    # 若存在权重文件，将参数加载到网络中
    if config.pre_trained:
```

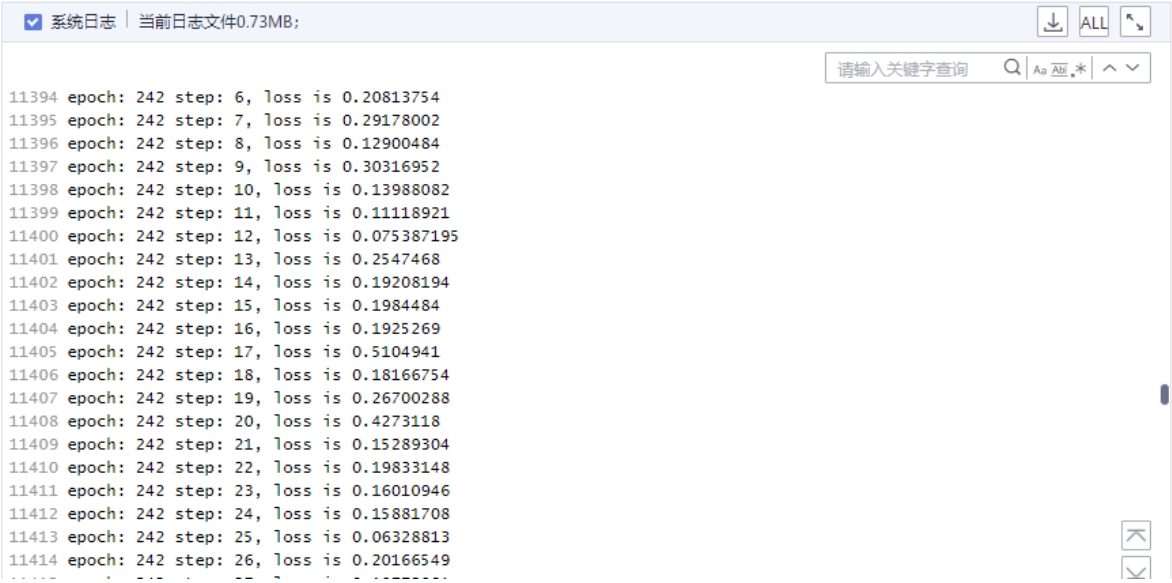
```
        load_param_into_net(network, load_checkpoint(config.pre_trained))

    # 定义学习率调整策略
    if config.lr_scheduler == 'exponential':
        lr = warmup_step_lr(config.lr,
                            config.lr_epochs,
                            config.steps_per_epoch,
                            config.warmup_epochs,
                            config.max_epoch,
                            gamma=config.lr_gamma,
                            )
    elif config.lr_scheduler == 'cosine_annealing':
        lr = warmup_cosine_annealing_lr(config.lr,
                                        config.steps_per_epoch,
                                        config.warmup_epochs,
                                        config.max_epoch,
                                        config.T_max,
                                        config.eta_min)
    elif config.lr_scheduler == 'step':
        lr = lr_steps(0, lr_init=config.lr_init, lr_max=config.lr_max,
warmup_epochs=config.warmup_epochs,
                      total_epochs=config.max_epoch, steps_per_epoch=batch_num)
    else:
        raise NotImplementedError(config.lr_scheduler)

    # 定义优化器函数
    opt = Momentum(params=get_param_groups(network),
                   learning_rate=Tensor(lr),
                   momentum=config.momentum,
                   weight_decay=config.weight_decay,
                   loss_scale=config.loss_scale)
    #定义损失函数，初始化模型参数
    if config.dataset == "flower_photos":
        loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
        model = Model(network, loss_fn=loss, optimizer=opt, metrics={'acc'},
                      amp_level="O2", keep_batchnorm_fp32=False,
loss_scale_manager=None)

    # 定义回调函数
    time_cb = TimeMonitor(data_size=batch_num)
    loss_cb = LossMonitor()
    callbacks = [time_cb, loss_cb]
    if config.rank_save_ckpt_flag:
        ckpt_config =
CheckpointConfig(save_checkpoint_steps=config.ckpt_interval *
config.steps_per_epoch,

keep_checkpoint_max=config.keep_checkpoint_max)
        save_ckpt_path = os.path.join(config.outputs_dir, 'ckpt_' +
str(config.rank) + '/')
        print(save_ckpt_path)
        ckpt_cb = ModelCheckpoint(config=ckpt_config,
                                  directory=save_ckpt_path,
                                  prefix='{}'.format(config.rank))
        callbacks.append(ckpt_cb)
```

```
#进行模型训练
model.train(config.max_epoch, dataset, callbacks=callbacks)
```

在Modelarts上训练，训练页面设置如下：



训练得到如下文件：



4.4.5.2 模型评估

使用测试数据集，使用`mindspore.Model.eval`接口进行评估，`eval`接口参数如下：

- `valid_dataset`：评估模型的数据集。
- `callbacks`：评估过程中需要执行的回调对象或回调对象列表。
- `dataset_sink_mode`：数据是否直接下沉至处理器进行处理。

`model.eval` 会返回一个字典，里面是传入metrics的指标和结果。

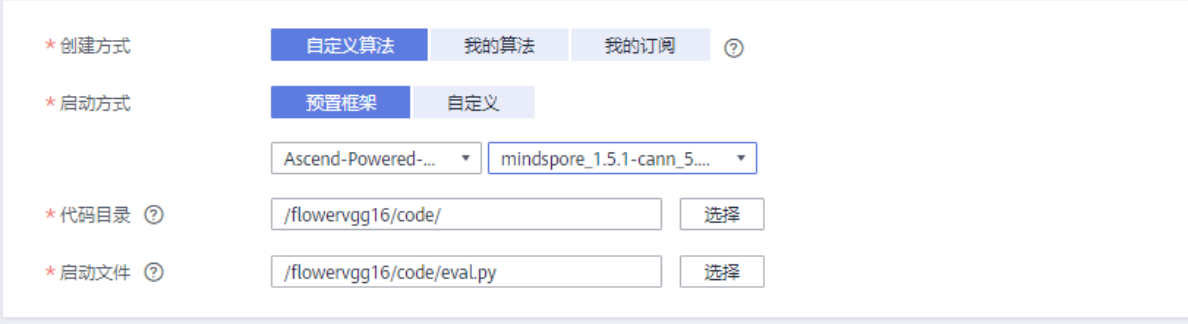
评估文件对应为（./code/eval.py），相关代码如下：

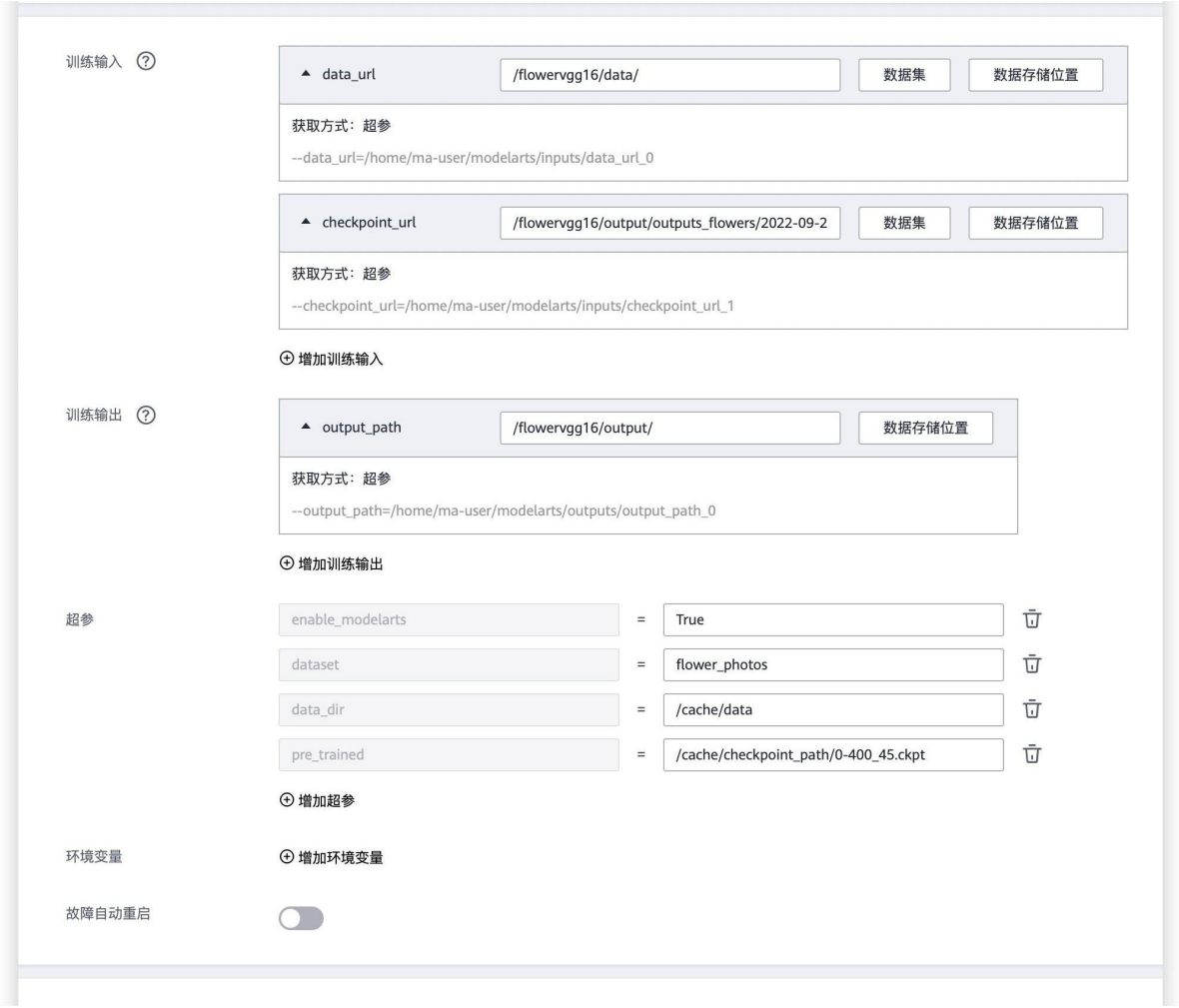
```
if config.dataset == "flower_photos":
        net = vgg17(num_classes=config.num_classes, args=config,phase="test")
        loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
        model = Model(net, loss_fn=loss, metrics={'acc'})
        param_dict = load_checkpoint(config.pre_trained)
        load_param_into_net(net, param_dict)
        net.set_train(False)
        dataset = vgg_create_dataset(config.data_dir, config.image_size,
config.per_batch_size, training=False)
        res = model.eval(dataset)
        print("result: ", res)
```

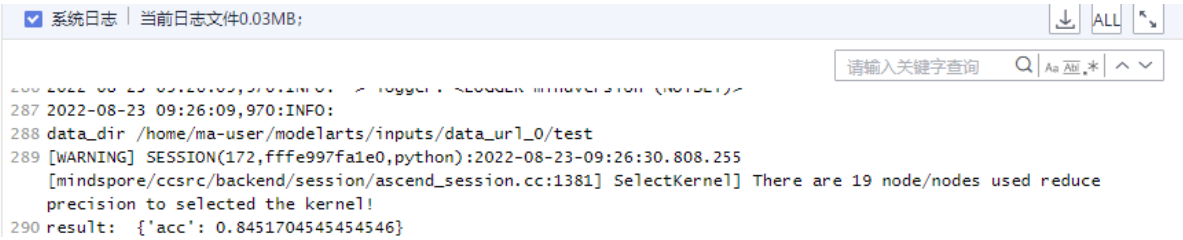
在Modelarts上训练，评估页面设置如下：

需要在训练输入中指定权重文件和数据集文件的目录，其中权重文件需要指定到ckpt存放的文件夹，

然后pre_trained超参 = /cache/checkpoint_path/权重文件名.ckpt







4.4.6 模型推理与部署（了解）

训练好的模型需要在更多的数据集进行推理和更多平台上进行部署。我们在训练中可以添加检查点（CheckPoint）用于保存模型的参数，以便进行推理及再训练使用。如果想继续在不同的硬件平台上做推理，可通过网络和Checkpoint格式文件生成对应的MindIR文件。

我们将花卉测试集送入训练好的模型执行推理。MindIR是MindSpore框架实现跨平台部署的文件格式，可以将Ascend AI处理器、GPU、CPU等不同平台上训练得到的Checkpoint导出为同一的MindIR格式。MindIR通过统一的表达定义了网络的逻辑结构和算子属性，将MindIR和硬件平台解耦，实现一次训练在不同的平台上部署。

4.4.6.1 导出MindIR文件

导出模型的对应文件为（./code/export.py），运行export.py文件，基于训练完成后的网络模型（即CKPT文件）转换为MindIR格式。

```
python export.py --config_path [YMAL_CONFIG_PATH] --ckpt_file [CKPT_PATH] --
file_name [FILE_NAME] --file_format [FILE_FORMAT]
```

注：batch_Size需设置为1，目前花卉数据集仅支持batch_Size为1的推理，即每次只进行一张图片的推理。

export.py的核心代码如下：

```
    # 定义网络
    if config.dataset == "flower_photos":
        net = vgg17(num_classes=config.num_classes, args=config)
    # 加载训练好的权重文件
    load_checkpoint(config.ckpt_file, net=net)
    net.set_train(False)
    # 定义网络输入
    input_data = Tensor(np.zeros([config.batch_size, 3, config.image_size[0],
config.image_size[1]]), mstype.float32)
    # 导出MindIR文件
    export(net, input_data, file_name=config.file_name,
file_format=config.file_format)
```

4.5 评分指标

70分：能够填写代码空缺部分，在Modelarts平台上完成模型训练和评估，输出权重文件，且训练和评估均在Modelarts上运行成功。

90分：能够填写代码空缺部分，在Modelarts平台上完成模型训练和评估，评估精度达到80%以上。

100分：能够填写代码空缺部分，在Modelarts平台上完成模型训练和评估，评估精度达到85%以上。

4.6 实验思考

- 对比开启混合精度前后，模型运行速度的快慢，并调研在给定数据集和不改变网络结构的前提下，提升模型训练速度的方法